DF 2100 - 01 – Rapid Prototyping

Spring 2026

Group ABS Absurdists:

Yuchen Xing, Eunchan Park, Daniel Bashirov, Caroline Caveness

Introduction

What is a Syringe Pump?

A syringe pump is a stepper motor driven pump designed to smoothly and accurately deliver measured amounts of liquid from a syringe to another object. At its core, a syringe pump is comprised of a linear rod, lead screw, a stepper motor to drive the lead screw, and securing hardware (lead screw collars, ball bearings, screws, nuts, flexible coupler, and bolts), all of which are responsible for the syringe pump's motion, given the correct power supply. These parts work together in keeping the materials in one place as a syringe is pushed forward, driven by the lead screw's motion.

Syringe pumps are most popular in laboratory and medical settings, as they are easy to keep sterile during syringe swaps; they are very accurate when it comes to pumping fluid, and their motion is smooth as opposed to pulsed. All of these details are important, especially since syringe pumps are most used to deliver nutrients, drugs, or blood to patients ("What Is a Syringe Pump? Learn the Types, Functions, & Uses," 2018).

Many syringe pumps support both infusion and withdrawal, and some can accommodate multiple syringes simultaneously, enabling incremental flow rates while still maintaining pulseless delivery. For the purposes of this project, only one syringe was implemented, which introduces the limitation of interrupted fluid delivery once the syringe is empty. In higher end situations, syringe pumps would have the ability to alert hospital staff when the syringe is almost empty—a feature that has been simulated in this project with an LCD screen, which displays how much time is remaining before the syringe pump is empty.

While syringe pumps require parts that are easily achievable, commercial variants can still cost several hundred dollars. Thus, designing and 3D printing the parts to a syringe pump, while implementing programmed electronics and buying the other necessary equipment (lead screws, securing hardware, motor, power supply, etc.) is a cost-effective alternative.

Design Requirements

In designing this linearly actuated syringe pump, there were several design constraints to address. The overarching goal for the enclosure, modeled in Fusion 360, was to keep it as compact, space-efficient, and cost-effective as possible, which was also pursued as an extra credit objective. Parts inside of the compartment were stacked, and walls were made as thin as possible to optimize the design. Further, certain regions, such as the area surrounding the power supply, were given additional clearance to account for wiring. The enclosure also had to protect all electrical components—the Arduino, driver, power supply, and associated wiring—from both the user and potential liquid exposure, so no holes were placed on the top surface, and any side or back holes were kept as small as possible.

The fit of the components into the enclosure walls was also carefully considered part by part, where buttons with nuts were designed with a transition fit so that they could be seated comfortably and then secured by tightening the nut, and parts without a nut (such as the indicator light or the Arduino USB port) were given a snap fit, held in place by the print walls. The Arduino USB port was given particular attention, as it was required to allow reprogramming of the Arduino without needing to open the enclosure, which would be screwed shut to create a

watertight and secure enclosure. To prevent the power bank from shifting during operation, a shallow 2 mm divot was incorporated into the floor of the enclosure, providing just enough to keep it seated. The syringe itself is held by thin 3D printed arms, designed with a comfortable sliding fit, loose enough to allow quick syringe swaps. All internal components were required to be fixed in place using fasteners or engineered snap fits: adhesives, glue, and hammering were explicitly not permitted, except for the linear bearing, which was press-fit into the carriage.

Wiring organization was also a significant design consideration. The project required all external components (buttons, lights, etc.) to use custom-length wires rather than pre-made jumper wires, with proper stripping, soldering, heat shrink, crimping and securing with DuPont connectors. All of the wires paired with buttons had to be long enough to run from the breadboard/Arduino to their respective locations without pulling. Jumper wires were only permitted for breadboard-to-breadboard or breadboard-to-Arduino connections.

To reduce confusion during assembly and debugging, wiring on the breadboard was kept as compact and tidy as possible, and all wiring was color-coded according to the prescribed scheme: black for AC live, white for neutral, green for earth/ground, black for DC ground, yellow for 5V, orange for 12V, red for 24V, and any other color for signal lines. This standard made it straightforward for multiple team members to work on the electronics without risk of mixing up voltage rails.

Operating the Pump

The following instructions outline the step-by-step process for operating the syringe pump. Before beginning, it must be ensured that the syringe pump is on a flat, stable surface, and that an appropriate syringe (10 mL or 20 mL for this specific project) is on hand and filled with fluid.

1. Choose a syringe, either 10 mL or 20 mL for this specific syringe pump. Do not fill it with liquid.
2. Open the Arduino code and confirm that the following variables are set to the desired parameters: *syringe_diameter_mm* (16.5 mm for a 10 mL syringe, or 21.15 mm for a 20 mL syringe), *syringe_mL* (10 or 20, must match the diameter variable above), and *flow_rate_mL_min* (a higher value corresponds to a faster flow rate).
3. Connect the Arduino to the computer using a USB to USB-C or USB to USB-A cable. The Arduino female connector is located in the side of the enclosure away from the group of other buttons. In the Arduino IDE, press the checkmark icon in the upper righthand corner to verify the code, then press the arrow next to it to push the code to the Arduino.
4. Once the upload is complete, disconnect the Arduino cable from both the Arduino and the computer.
5. Before loading the syringe, plug the power supply into the wall. Observe the pump's behavior and use this moment to verify that the carriage speed looks appropriate for the intended flow rate:
    a. If the carriage begins moving immediately, observe the speed as it runs.
    b. If the indicator light is yellow (paused), press the start/pause button to resume and observe the speed.

    c. If the indicator light is red (carriage fully compressed), press the green button to move the carriage backward, un-pause if needed, and then observe the speed.
6. Use the green and red buttons to move the carriage backward or forward (counterclockwise and clockwise, respectively) until the syringe can be set between the end support and the carriage, nested in the arms. If the syringe is not fully extended, use the green button to move the carriage backward until the syringe is fully extended.
7. When the syringe is fully extended, take the syringe out and fill it with your desired fluid.
8. Slide the syringe into the retaining arms. Once seated, the pump is ready to run. Press the start/pause button to begin the movement of the pump.

Methods - Syringe Pump Contents

This linearly actuated syringe pump project required the implementation of Computer Aided Design (CAD) with Fusion 360, electrical prototyping with an Arduino, a stepper driver, a breadboard, and various external components, and physical assembly with a lead screw, a linear rod, a coupler, a stepper motor, an aluminum bar extrusion, and various securing hardware components.

Functionality - CAD

The syringe pump operates as follows:

The plunger of the syringe is attached to the carriage, and the end of the barrel is attached to the end support. When the pause/start button is pressed, the button sends a signal to the Arduino board. Then, it processes the signal and sends commands to the motor, including the speed and direction of the rotation.

As the motor rotates, the lead screw transforms the rotational motion into linear displacement: the carriage, which is coupled to the lead screw with a nut, travels along the length of the screw as it spins. Therefore, when the carriage moves back and forth, the plunger moves correspondingly.

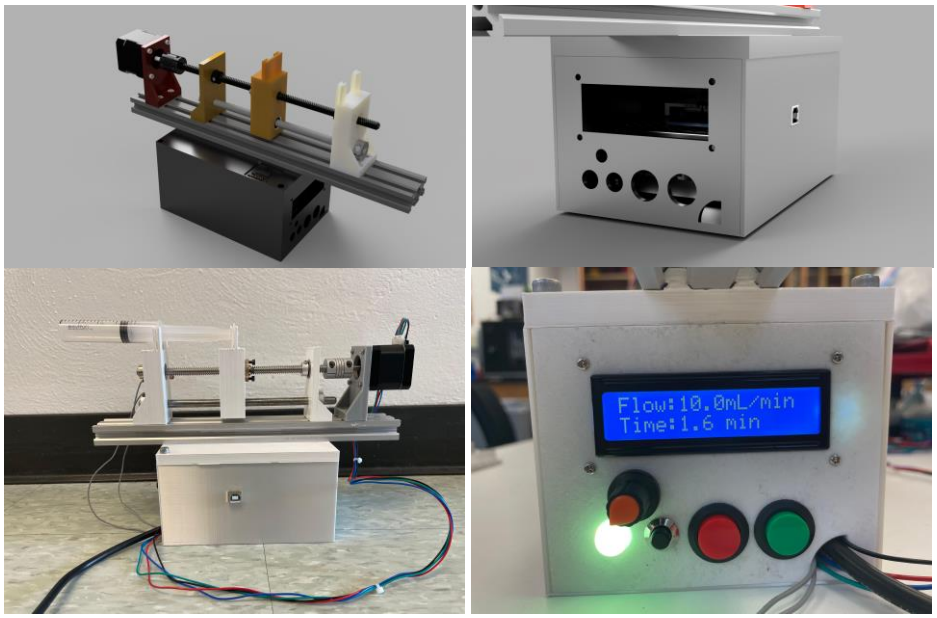


Figure 1. Top-Left: Full rendered view of the linear actuator and enclosure assembly. Top-Right: Close-up view rendering of the enclosure with internal components. Bottom Left/Bottom Right: Full real-life assembly of linear actuator and enclosure.

While prototyping for the carriage, end supports, enclosure and modular wall for this linear actuator, each component had small issues that caused the need for reprints. See the pictures below for a comparison of the original design and the final design of the end support and the carriage.

Carriage and End Supports

All necessary dimensions were measured with a caliper. The key differences between the two syringes are the length, size of the plungers, flanges, and the size of the cross-section. In order to fit syringes of multiple sizes, the carriage and end supports were designed to fit to the syringe of larger diameter (the 20 mL syringe). This way, any syringe smaller would have proper clearance. As for the length, two securing arms/slots were designed and implemented onto the carriage and end support, each ensuring that both syringes could be pushed to completion.

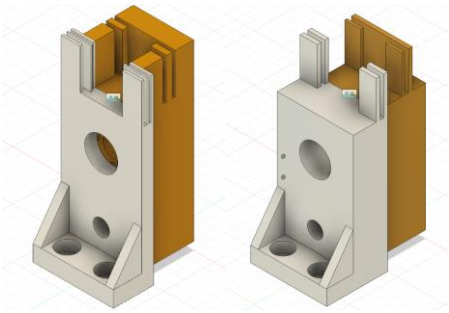


Figure 2. Rendering of carriage and end support in Fusion360. (Left) First iteration and (Right) final iteration of end support CAD design.

One of the encountered problems within the CAD design aspect was the overdesign of the carriage and one of the end supports (Figure 2). The syringes were observed to have many minor differences *in addition to* the volumes. The original design aimed to make everything compatible by assigning clear slots to each of the syringes, but this process led to unnecessary details and delayed the process of development. Later, when it was decided to implement both forward and backward movement in the linear actuator, the current design with the securing arms was created. This design allows the syringe to be pushed or pulled in either direction. In short, we solved it by adjusting the model in CAD to a more simplified design, prioritizing the larger syringe to accommodate both the smaller and larger sizes.

Modular Wall

The modular wall was designed to hold a start/pause button (Panel Mount Latching Push Button), two monetary push buttons, a tri-color LED (RGB common cathode LED), a 10k potentiometer, and a hole for exit wiring. Additionally, the wall was designed to slide into the enclosure so that it could be easily removed for purposes such as troubleshooting inner materials, or perhaps changing out the design in the future. Figure 3, displayed to the right, shows a 3D rendered model of the modular wall, with all of the proper holes for the components.

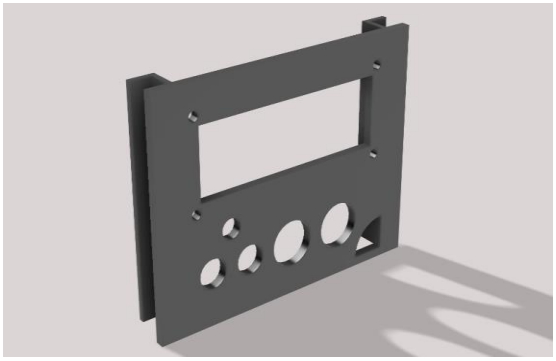


Figure 3: 3D Rendering of Modular Wall in Fusion360

Two prints of the modular wall were produced, where the first print had issues with sizing and fit. For one, the diameter of the hole for the start/pause button was too small (9.7 mm), despite being given a .2 mm clearance from 9.5 mm. This hole was enlarged to 10.12 mm, after the button was pressed through the hole meant for the light, and the slide fit was more loose and perfect for a component that was backed by a nut.

Secondly, the diameter of the hole for exit wiring was also too small, with a quarter size hole holding a diameter of 8.5. This hole was increased to have a diameter of 12.05 as the wiring for the motor needed at least 10 mm to be threaded through, and the cord for the power supply needed a 6 mm diameter gap to occupy.

Lastly, the gaps that were meant to fit around the columns of the enclosure to form a slide fit were too narrow to perform a slide fit. This was mostly caused by the diameter of the heatset inserts, which also enlarged the dimensions of the columns after inserting. This enlargement was not accounted for, so the gap was altered from a 7.70 mm gap to an 8.10 mm gap. Figure 4 displays front and top-down views of the first and second iterations, and the caption explains which components changed in the process.

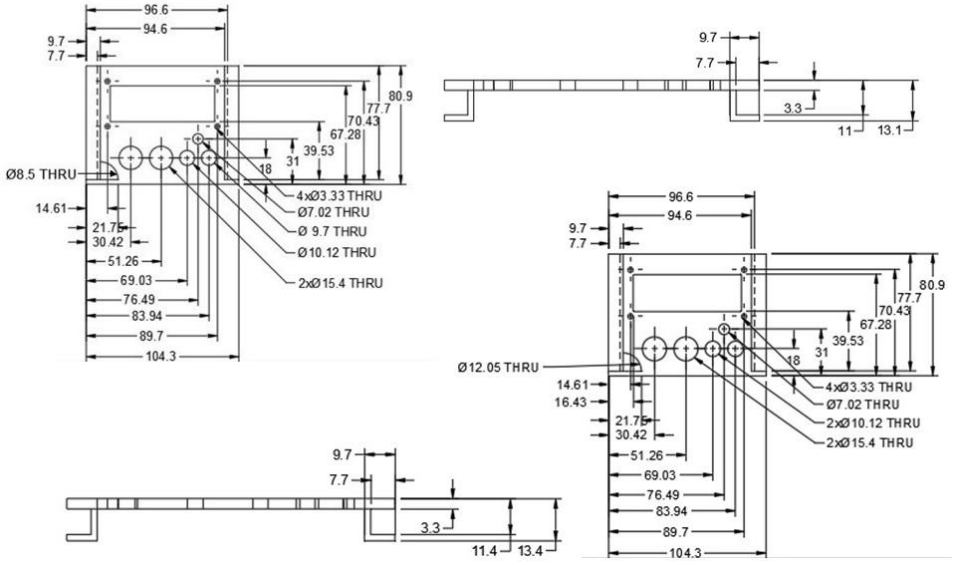


Figure 4. Top left: The first iteration front view with a smaller pause/start button through hole (9.7 mm diameter), and wiring through hole (8.5 mm diameter). Top right: first iteration top-down view, with a smaller gap (7.7 mm, show as 11 mm). Bottom right: second iteration front view with the start/pause button (10.12 mm THRU) and the wiring hole (12.05 mm THRU). Bottom left: second iteration enlarged slip-fit gap (8.1 mm, shown as 11.4 mm).

Enclosure

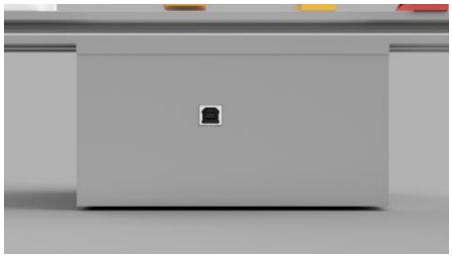


Figure 5. 3D Rendered Model of the Enclosure, Displaying the Side with the Arduino USB Hole

The enclosure's primary purpose is the following: protecting the internal electronic components from physical impacts and water intrusion that risks safety. Specifically, rather than a snuggle or snap fit, the screw design ensures the gap between PLA enclosure and lid is watertight. This screw-in design was made possible by the application of M5 heatset inserts into the three rectangular columns (7.5mm × 7.5mm × 78mm), each with a centered circular hole (diameter = 6.5mm). Despite missing a screw for one of the corners due an insufficient space for a column, the three screws were able to provide structural stability and watertightness. Additionally, provided that the lid fully covers the top surface of the enclosure, the modular wall fitted below the lid provided the necessary physical stability of the fully assembled enclosure.

However, in the first and second versions of the enclosures, one of the screw holes was too close to the edge of the lid. Consequently, due to limited precision of FFF printing and slicing of a smooth curve, a portion of the screw hole was not printed. However, despite the gap between the hole's edges, the remaining, well-printed screw hole edges were able to cover the entirety of the top surface and endure the physical load for watertightness.



Figure 6. Small gap in the screw holes on the lid. No major changes in the CAD design as no structural stability or waterproofness were compromised.

As for the first prototype of the modular wall, there had to be a larger hole for the power supply and motor wires from the inside of the enclosure. Additionally, the slide-fit of the modular wall into two of the rectangular corner columns of the enclosure had to be adjusted. While the design offset of the fit theoretically enables slide-fit, the application of the heatset inserts expanded the heatset insert hole of the columns. Ultimately, the final prototype of the modular wall had a quarter circle shaped wire hole with radius of 12.05mm and 0.3mm offsets for the slide-fit.

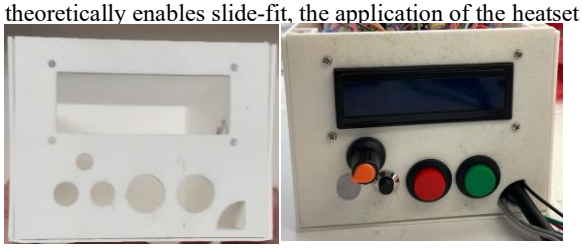


Figure 7 Modular Wall Version 1(Left) and Version (2). Increased wire hole size and slide-fit offset in CAD

Finally, while creating the modular wall, it was realized that there would not be enough space to include a hole for the Arduino USB to be implemented—at least not in a space-efficient and comfortable fashion. The enclosure had already been printed once, but because of this detail, the hole for the Arduino USB had to be implemented in the CAD and reprinted for the next use. Since the Arduino was facing to the right on the inside of the enclosure to be easily nested against the breadboard, the hole for the USB was created in the right side of the enclosure—with the hole's bottom right corner beginning 89 mm by 43.50 mm (L x W) from the bottom back corner on the same side, which is where the power bank would have been located. Figure 8 displays the dimensional drawings of the enclosure before the hole was added and after the hole was added.

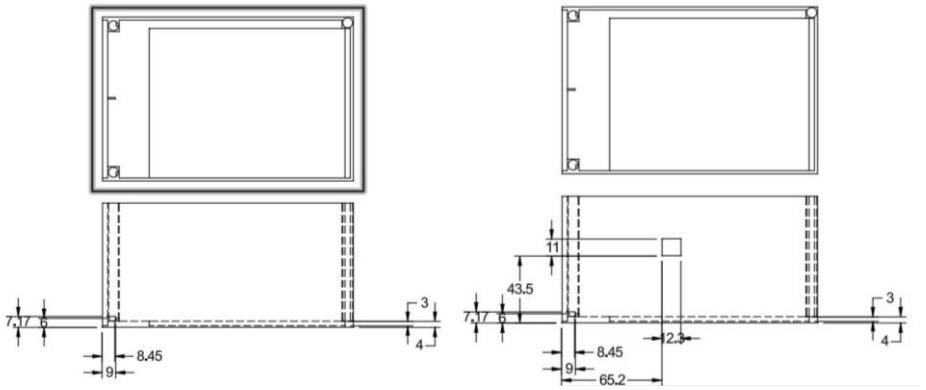


Figure 8. (Left) Top and side view of the enclosure before the Arduino USB hole was implemented. (Right) Top and side view of the enclosure after the Arduino USB hole was implemented. Drawn in Fusion360.

Enclosure Lid

The lid serves two purposes. Firstly, it has an extruded tray on the top, which provides a snap fit for the aluminum extrusion. The shape of the tray is designed to take the shape of the aluminum extrusion and add a 1 mm offset. Figure 9 provides a detailed view of the implementation.

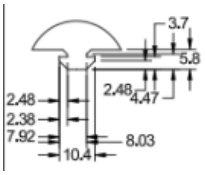


Figure 9. The shape of the slot, located at the top of the lid. Drawn in Fusion360.

Secondly, the lid provides protection to the vulnerable electronics in the enclosure from any liquid from the above. It attaches to the enclosure with three holes for the screws. And those screws fit into the heat-set inserted holes on the enclosure.

It only took one print for the lid, but the print wasn't perfect. In fact, one of the corners is damaged, and the screw hole is incomplete. The print was closely examined to ensure the damage doesn't affect functionality. Even though about half of the wall for the screw is damaged, it still provides alignment and enough support for the screws, so the damage was accepted.



Figure 10. "No BS ABS Absurdists" Team Logo Printed into the Lid of the Enclosure

As an additional, creative element, the ABS Absurdists Team created their own logo, deeming themselves the "No BS ABS Absurdists" with a cat to accompany.

批注 [EP1]: Hey Yuchen. For this part, Caroline and I both think that the drawing of the slide slot and the full drawing of the lid should be one figure together. I will work on formatting figures. Thanks!

批注 [EP2]: Ermmm... I will leave section to you. I think the delay of editing is not working out well.

Electronics

Control Inputs

Three input devices and one indicator output connect back to the Arduino Uno. The latching button is the main start and pause control. Once latched, the carriage drives forward at the commanded flow rate to dispense fluid through the syringe. Pressing it again unlatches the circuit and the carriage stops. Two momentary buttons handle syringe loading. The left button jogs the carriage forward and the right jogs it backward, each active only while held. This arrangement positions the plunger against whatever syringe is loaded without requiring the lead screw to be turned by hand. Jog speed runs at the same configured step rate as dispense, fast enough to load a syringe in roughly five seconds and slow enough to avoid overshoot.

Limit Switch

The limit switch sits at the end of the lead screw travel. When the carriage runs the plunger to the bottom of the syringe and trips the switch, the motor cuts and the LED turns red to signal that the syringe is empty. The switch is wired normally open with the Arduino's internal pullup enabled, so a triggered state reads HIGH on the input pin.

Indicator LED

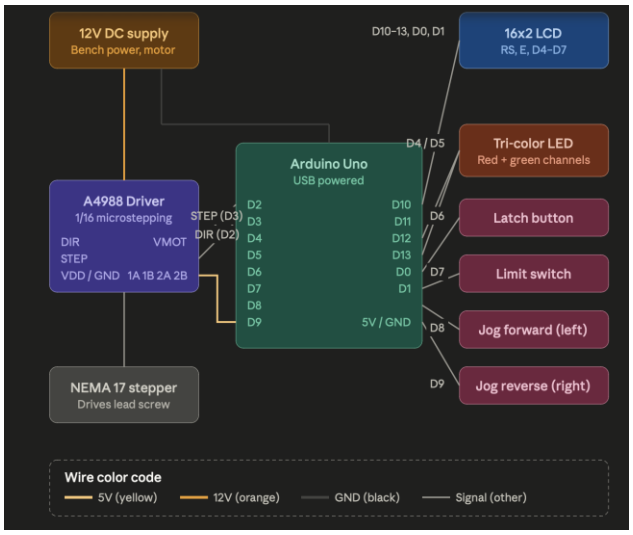
The tri-color LED is a common-cathode RGB module with the red and green channels each tied to their own PWM pin. Green indicates active dispensing or jogging. Yellow marks the paused state and is produced by driving both the red and green channels HIGH at the same time, which mixes to yellow at the LED. That state appears whenever the latch is open and no jog button is held, showing that the system is powered and idle. Red, triggered by the limit switch, indicates that the carriage has reached the end of travel and dispensing has stopped.

Wiring and Color Code

Wiring follows a project color code so that an operator can identify the purpose of any wire without tracing it. Yellow carries 5V, orange carries 12V, and red carries 24V. Black is ground or common across both the AC and DC sides. Logic signals, including the step and direction lines to the driver, the three button inputs, the limit switch, and the LED control lines, are run in colors outside the reserved voltage palette. All wiring from the Arduino and breadboard out to external components, the motor, the limit switch, the buttons, and the LED, is built from custom-length harnesses cut from spool wire, stripped, soldered, and heat-shrunk at the terminations. Jumper wires appear only on breadboard-to-breadboard or breadboard-to-Arduino connections.

Power

The Arduino is powered over USB, which keeps the programming port accessible without opening the enclosure. A DC bench supply provides 12V to the A4988 driver's VMOT input and from there to the NEMA 17 stepper. The Arduino logic ground and the DC supply ground are tied together at the breadboard rail so the step and direction signals share a common reference with the driver. All power and driver components sit inside the enclosure with no top-surface penetrations. Wire pass-throughs for the motor, the switch, and the buttons are located on the side wall and sized just large enough for the harness.



Figure(10) Wiring

Code

Sketch Overview

The Arduino sketch performs three tasks. It converts the user-set flow rate into a step frequency for the motor. It polls the three buttons and the limit switch on every loop iteration to determine the system state. It also refreshes the LCD twice per second with the current flow rate and the time remaining until the syringe runs empty.

Configuration

Adjusting dispense behavior requires editing only two variables at the top of the sketch: flow_rate_mL_min for the target flow rate and syringe_diameter_mm for the inner diameter of the loaded syringe. Every other parameter is computed from those two inputs inside setup() and stored in steps_per_sec, which the main loop feeds to the stepper.

Main Loop and State Logic

The main loop reads all four inputs each cycle and selects the system state from their combination. When the left jog button is held on its own, the motor runs forward at the configured speed and the LED shows green. When the right jog button is held on its own, the motor runs in reverse at the same speed with the LED still green. A trip on the limit switch forces the motor to zero speed and switches the LED to red regardless of any other input, so the empty condition takes priority over every other state. When the latch button is engaged and no jog button is active, the motor runs in the dispense direction at the commanded flow rate with the

LED green. In every remaining case the motor is stopped and the LED is yellow, marking a powered, idle system waiting for input.

LCD Update

The LCD update is throttled by a millis() check against LCD_INTERVAL, set to 500 ms, so the screen refresh does not compete with step pulse generation in the main loop. On each refresh the code reads the stepper's current position, converts it back to millimeters of plunger travel using steps_per_mm, and from there computes the volume dispensed and the volume remaining. Time remaining follows as remaining volume divided by the commanded flow rate. Both the flow rate and the time remaining are written to the LCD on every refresh.

Motor Functionality and Logic

The stepper motor is driven using the AccelStepper library and configured in DRIVER mode, accepting step and direction signals on pins 3 and 2 respectively. Microstepping is set to 1/16 by pulling the MS1, MS2, and MS3 pins on the driver high, yielding an effective resolution of 200 × 16 = 3,200 steps per revolution. This level of microstepping was chosen to produce smooth, low-vibration motion, which is important for consistent and pulseless fluid delivery.

The theoretical maximum flow rate is governed by the maximum step frequency the driver and Arduino can reliably output when using the AccelStepper library, which is approximately 4,000 steps/sec on an Arduino Uno. Working backwards from this limit:

$$w_{max} = \frac{4000\ steps/\sec \times 60}{3200\ steps/rev} = 75\ rev/min$$

$$v_{max} = 75 \times 8.0 = 600\ mm/min$$

For the 10 mL syringe (diameter = 16.5 mm, Area = $213.8\ mm^2$*):*

$$V_{maxflowrate} = 600\frac{mm}{min} \times 213.8\ mm^2 \approx 128{,}280\frac{mm^3}{min} \approx 128.3\frac{mL}{min}$$

For the 20 mL syringe (diameter = 21.15 mm, Area = $351.6\ mm^2$*):*

$$V_{maxflowrate} = 600\frac{mm}{min} \times 351.6\ mm^2 \approx 210{,}960\frac{mm^3}{min} \approx 211.0\frac{mL}{min}$$

It should be noted that these are theoretical maximums. In practice, the actual maximum flow rate will be lower, as stepper motor torque drops off at high speeds and steps may be skipped before this frequency is reached.

The smallest dispensable volume is set by the displacement produced by a single step of the motor. With a lead screw pitch of 8 mm/rev and 3200 steps/rev, the linear displacement per step is:

$$\Delta x_{step} = \frac{8\ mm/rev}{3200\ steps/rev} = 0.0025\ mm/step$$

For the 10 mL syringe:

$$\Delta V_{10} = 0.0025 \times 213.8 \approx 0.534 \frac{mm^3}{step} \approx 0.000534\ mL/step$$

For the 20 mL syringe:

$$\Delta V_{20} = 0.0025 \times 351.6 \approx 0.879 \frac{mm^3}{step} \approx 0.000879\ mL/step$$

These represent the theoretical resolution minimum—the smallest increment of fluid the pump can dispense per step for each syringe size. The larger cross-sectional area of the 20 mL syringe means that each step displaces a greater volume, making the 10 mL syringe the more precise option for low flow rate applications.

Example Calculation: Converting Flow Rate to Steps per Second

The following example demonstrates how a commanded flow rate is converted to a motor speed in steps per second, using the 10 mL syringe at a flow rate of 10.0 mL/min.

Given:

- Syringe diameter: $d = 16.5$ mm
- Flow rate: $V_{flowrate} = 10.0$ mL/min
- Lead screw pitch: $L = 8.0$ mm/rev
- Steps per revolution: $N = 3200$ steps/rev

Step 1. Syringe Cross Sectional Area

$$A = \left(\frac{16.5}{2}\right)^2 \pi \approx 213.8\ mm^2$$

Step 2. Convert Flow rate to mm^3/min

$$V_{flowrate} = 10.0 \frac{mL}{min} \times 1000 \frac{mm^3}{mL} = 10000\ \frac{mm^3}{min}$$

Step 3. Plunger Linear Speed

$$v = \frac{V_{flowrate}}{A} = \frac{10000\ mm^3/min}{213.8\ mm^2} \approx 46.77 \frac{mm}{min}$$

Step 4. Convert Linear Speed to Revolutions per Min

$$w = \frac{v}{L} = \frac{46.77\ mm/min}{8\ mm/rev} \approx 5.85 \frac{rev}{min}$$

Step 5. Convert to Steps per Second (Frequency)

$$f = \frac{w \times N}{60} = \frac{5.85 \times 3200}{60} \approx 312 \frac{steps}{sec}$$

This value is passed directly into stepper.setSpeed() in the Arduino code.

Table 1. *Table of Variables in the Arduino Code*

Variable	Value/Type	Description
syringe_diameter_mm	float	Inner diameter of the syringe in mm. Must be set to match the syringe in use (16.5 mm for 10 mL, 21.15 mm for 20 mL).
syringe_mL	float	Volume of the syringe in mL (10 or 20). Must match the diameter variable above.
flow_rate_mL_min	float	Commanded flow rate in mL/min. Primary user-adjustable parameter for controlling delivery speed.
steps_per_rev	int	Total steps per revolution after microstepping (200 base × 16 microstep = 3,200).
lead_mm_per_rev	float	Linear travel of the lead screw per full revolution (8.0 mm/rev).
steps_per_sec	float	Computed motor speed in steps/sec. Derived from the above variables and passed to the stepper driver.
red_led_pin	int	Pin 4. Controls the red channel of the tri-color LED.
green_led_pin	int	Pin 5. Controls the green channel of the tri-color LED.
latch_button_pin	int	Pin 6. Input from the latching start/pause button (INPUT_PULLUP).
limit_switch	int	Pin 7. Input from the limit switch that detects when the syringe is fully depleted (INPUT_PULLUP)
left_button_pin	int	Pin 8. Input from the left jog button for moving the carriage leftward (INPUT_PULLUP).
right_button_pin	int	Pin 9. Input from the right jog button for moving the

		carriage rightward (INPUT_PULLUP).
autoRunning	bool	Tracks whether the pump is currently in automatic run mode.
lastLatchState	bool	Stores the previous state of the latch button to detect state changes.
steps_per_mm	float	Conversion factor from steps to millimeters of carriage travel, used for LCD volume calculations.
lastLcdUpdate	unsigned long	Timestamp of the last LCD refresh, used to throttle display updates.
LCD_INTERVAL	unsigned long	Minimum time in milliseconds between LCD updates (500 ms). Prevents excessive writes that could affect motor timing.

Appendix

Arduino Code

```
// Including libraries for stepper motor motion and LCD display
#include <AccelStepper.h>
#include <LiquidCrystal.h>

// Initialize LCD with pin connections (RS, E, D4, D5, D6, D7)
LiquidCrystal lcd(12, 11, 0, 1, 13, 10);

// Establishing pin variables
int red_led_pin = 4;
int green_led_pin = 5;
const int latch_button_pin = 6;  // start/pause button
const int left_button_pin = 8;   // jog carriage left
const int right_button_pin = 9;  // jog carriage right
int limit_switch = 7;
bool autoRunning = false;        // tracks whether pump is in automatic run mode
bool lastLatchState = HIGH;      // stores previous latch button state to detect changes

// Establishing adjustable variables
const float syringe_diameter_mm = 21.15;  // can change syringe diameter (16.5 mm for 10 mL, 21.15 mm for 20 mL)
const float syringe_mL = 20.0;            // can adjust between 10 or 20 - must adjust syringe diameter as well
const float flow_rate_mL_min = 10.0;      // can change flow rate

const int steps_per_rev = 200 * 16;       // steps per rev acquired from datasheet - all MS pins pulled high for 1/16 microstep
const float lead_mm_per_rev = 8.0;        // lead screw travel per revolution in mm
float steps_per_sec = 0.0;
```

```
float steps_per_mm = (steps_per_rev / lead_mm_per_rev) * 5.0;  // conversion factor from steps to mm, used for LCD volume calculation


// LCD update throttling - prevents writing to LCD too frequently which can affect motor timing
unsigned long lastLcdUpdate = 0;
const unsigned long LCD_INTERVAL = 500;  // minimum ms between LCD refreshes


AccelStepper stepper(AccelStepper::DRIVER, 3, 2);  // stepper in DRIVER mode


void setup() {


  // Calculating and setting speed
  float radius_mm = syringe_diameter_mm / 2.0;               // finding radius from diameter
  float area_mm2 = 3.14159 * radius_mm * radius_mm;          // finding cross-sectional area of syringe
  float flow_mm3_min = flow_rate_mL_min * 1000.0;            // converting mL to mm^3
  float plunger_speed_mm_min = flow_mm3_min / area_mm2;      // converting to linear plunger speed in mm/min
  float revs_per_min = plunger_speed_mm_min / lead_mm_per_rev; // finding revolutions per min on lead screw
  steps_per_sec = (revs_per_min * steps_per_rev) / 60.0;     // converting to steps per second for the motor to use
  steps_per_sec *= 5.0;                                      // scaling factor to match physical carriage behavior


  stepper.setMaxSpeed(steps_per_sec * 2);  // setting max speed with safety headroom for jogging
  stepper.setSpeed(steps_per_sec);


  // Establishing pin modes
  pinMode(red_led_pin, OUTPUT);
  pinMode(green_led_pin, OUTPUT);
```

```
  pinMode(latch_button_pin, INPUT_PULLUP);
  pinMode(limit_switch, INPUT_PULLUP);
  pinMode(left_button_pin, INPUT_PULLUP);
  pinMode(right_button_pin, INPUT_PULLUP);

  // Initialize LCD and show startup message
  lcd.begin(16, 2);
  lcd.print("Syringe Pump");
  delay(1500);
  lcd.clear();
}

void loop() {

  // Read all button and switch states at the start of each loop
  bool leftPressed = digitalRead(left_button_pin) == LOW;
  bool rightPressed = digitalRead(right_button_pin) == LOW;
  bool autoMode = digitalRead(latch_button_pin) == LOW;
  bool limitTriggered = digitalRead(limit_switch) == HIGH;

  // Jog left (carriage moves away from motor)
  if (leftPressed && !rightPressed) {
    stepper.setSpeed(steps_per_sec);
    stepper.runSpeed();
    analogWrite(green_led_pin, 255);  // green = running
    analogWrite(red_led_pin, 0);

  // Jog right (carriage moves toward motor)
  } else if (rightPressed && !leftPressed) {
    stepper.setSpeed(-steps_per_sec);
```

```
  stepper.runSpeed();
  analogWrite(green_led_pin, 255);  // green = running
  analogWrite(red_led_pin, 0);

// Limit switch triggered - syringe is empty, stop motor and show red
} else if (limitTriggered) {
  stepper.setSpeed(0);
  analogWrite(green_led_pin, 0);
  analogWrite(red_led_pin, 255);  // red = empty

// Auto mode active - pump running normally
} else if (autoMode) {
  stepper.setSpeed(-steps_per_sec);
  stepper.runSpeed();
  analogWrite(green_led_pin, 255);  // green = running
  analogWrite(red_led_pin, 0);

// Paused - no buttons pressed and limit not triggered
} else {
  stepper.setSpeed(0);
  analogWrite(green_led_pin, 255);  // yellow = paused (green + red)
  analogWrite(red_led_pin, 255);
}

// Update LCD every LCD_INTERVAL ms to display flow rate and time remaining
if (millis() - lastLcdUpdate >= LCD_INTERVAL) {
  lastLcdUpdate = millis();

  // Calculate volume dispensed and remaining based on carriage position
  long currentPos = stepper.currentPosition();
```

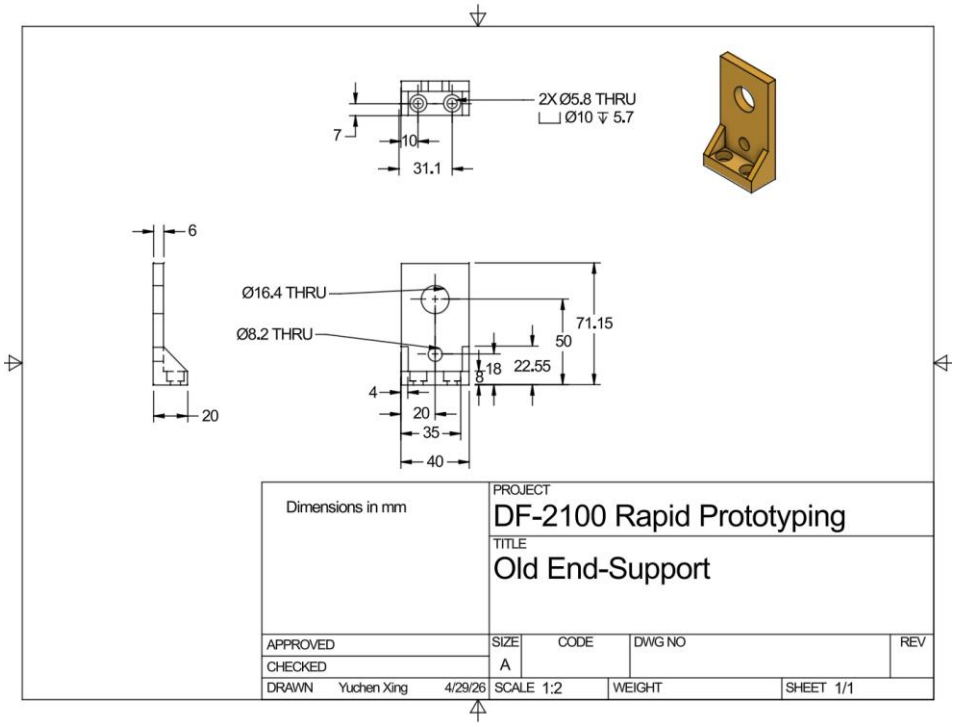
```
    float mm_traveled = abs(currentPos) / steps_per_mm;       // convert steps to mm traveled
    float radius_mm = syringe_diameter_mm / 2.0;
    float area_mm2 = 3.14159 * radius_mm * radius_mm;
    float vol_dispensed_mL = (mm_traveled * area_mm2) / 1000.0; // convert mm³ to mL
    float vol_remaining_mL = syringe_mL - vol_dispensed_mL;

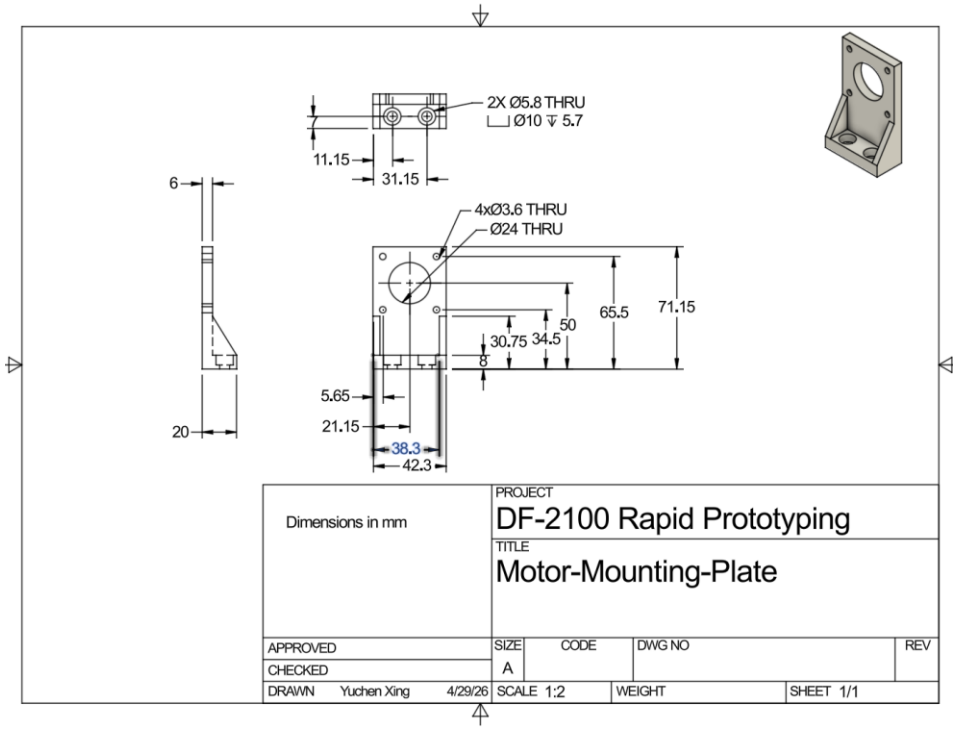
    if (vol_remaining_mL < 0) vol_remaining_mL = 0;  // clamp to zero to avoid negative display
    float time_remaining_min = vol_remaining_mL / flow_rate_mL_min;  // calculate time
remaining in minutes

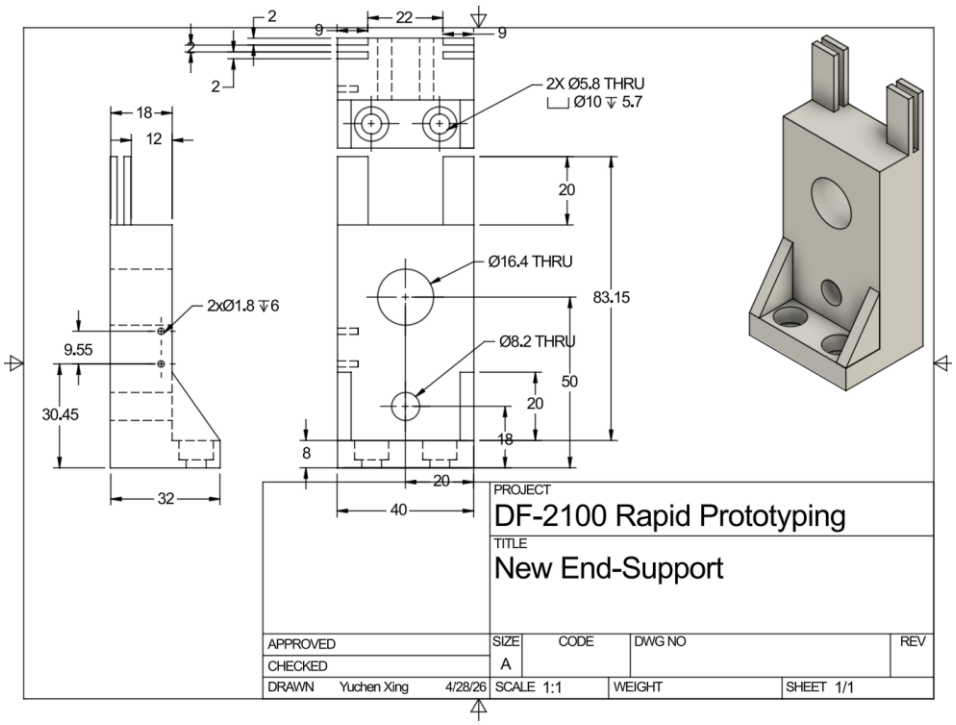
    // Write flow rate to top row of LCD
    lcd.setCursor(0, 0);
    lcd.print("Flow:");
    lcd.print(flow_rate_mL_min, 1);
    lcd.print("mL/min  ");

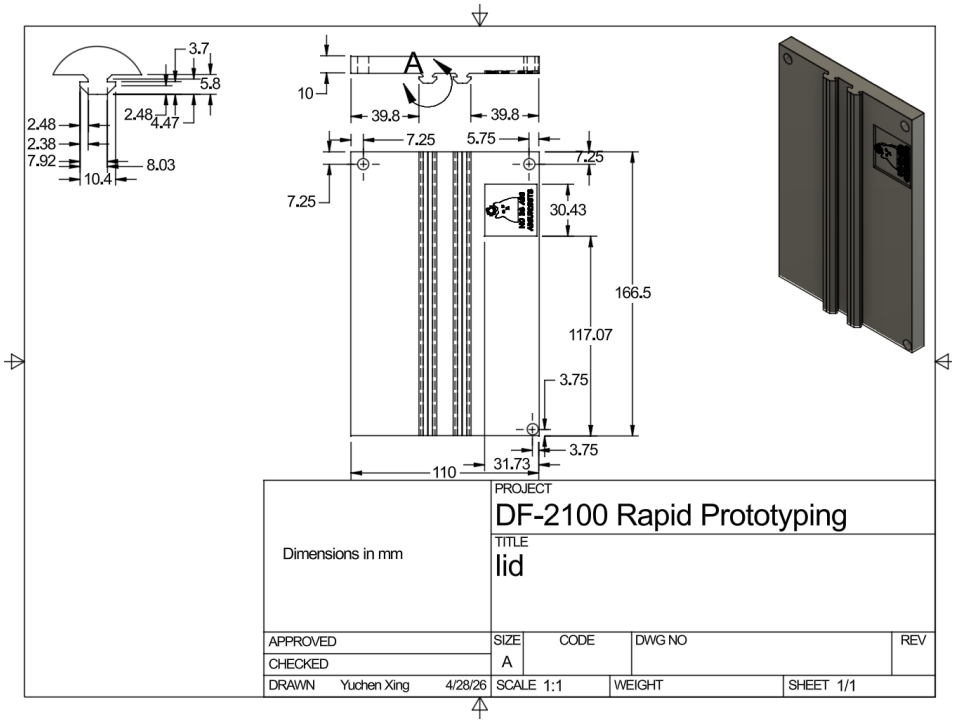
    // Write time remaining to bottom row of LCD
    lcd.setCursor(0, 1);
    lcd.print("Time:");
    lcd.print(time_remaining_min, 1);
    lcd.print(" min     ");
  }
}
```

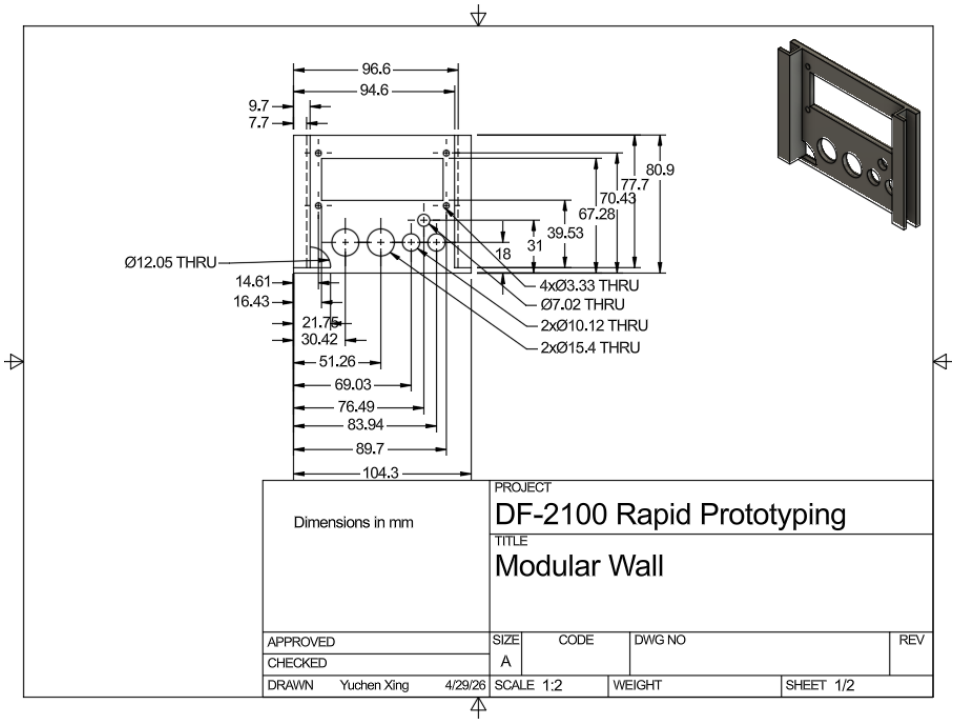
Dimensioned Drawings:

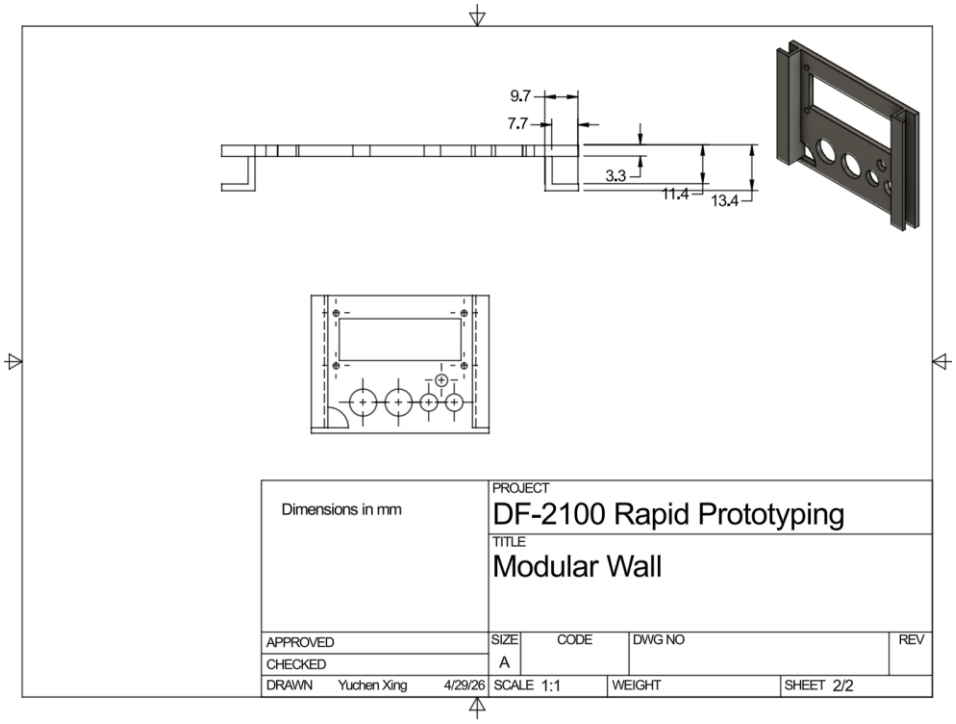

2X Ø5.8 THRU
Ø10 ↧ 5.7
7
10
31.1
6
Ø16.4 THRU
Ø8.2 THRU
71.15
50
18
22.55
8
4
20
35
40
20
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
Old End-Support
APPROVED
CHECKED
DRAWN Yuchen Xing 4/29/26
SIZE
A
CODE
DWG NO
REV
SCALE 1:2
WEIGHT
SHEET 1/1

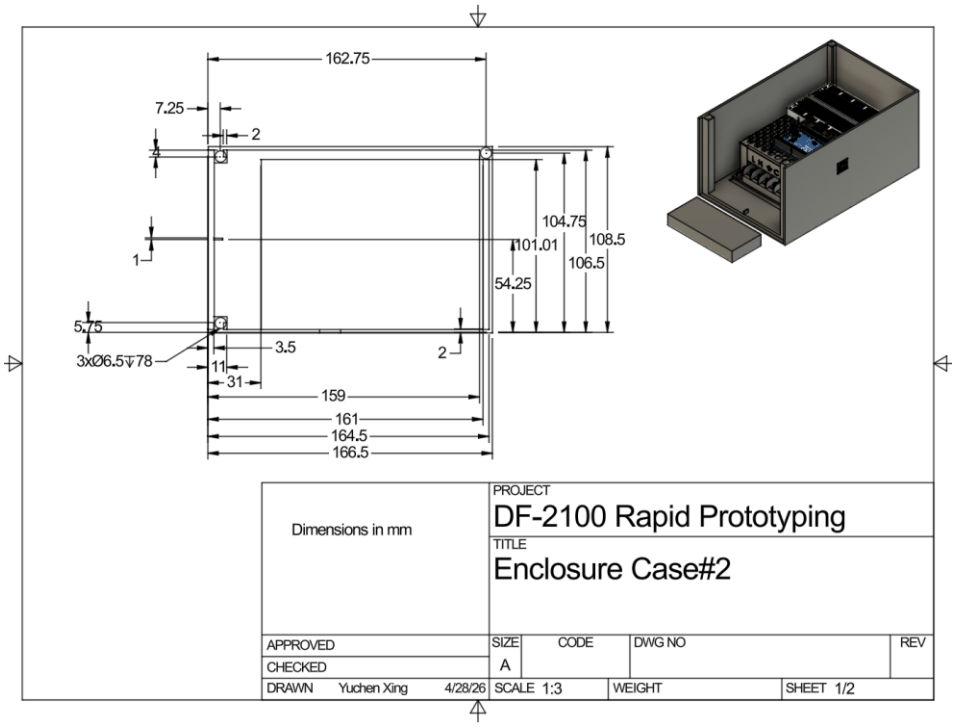

2X Ø5.8 THRU
Ø10 ↧ 5.7
11.15
31.15
6
4xØ3.6 THRU
Ø24 THRU
65.5
71.15
50
30.75
34.5
8
5.65
21.15
38.3
42.3
20
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
Motor-Mounting-Plate
APPROVED
CHECKED
DRAWN Yuchen Xing 4/29/26
SIZE
A
CODE
DWG NO
REV
SCALE 1:2
WEIGHT
SHEET 1/1

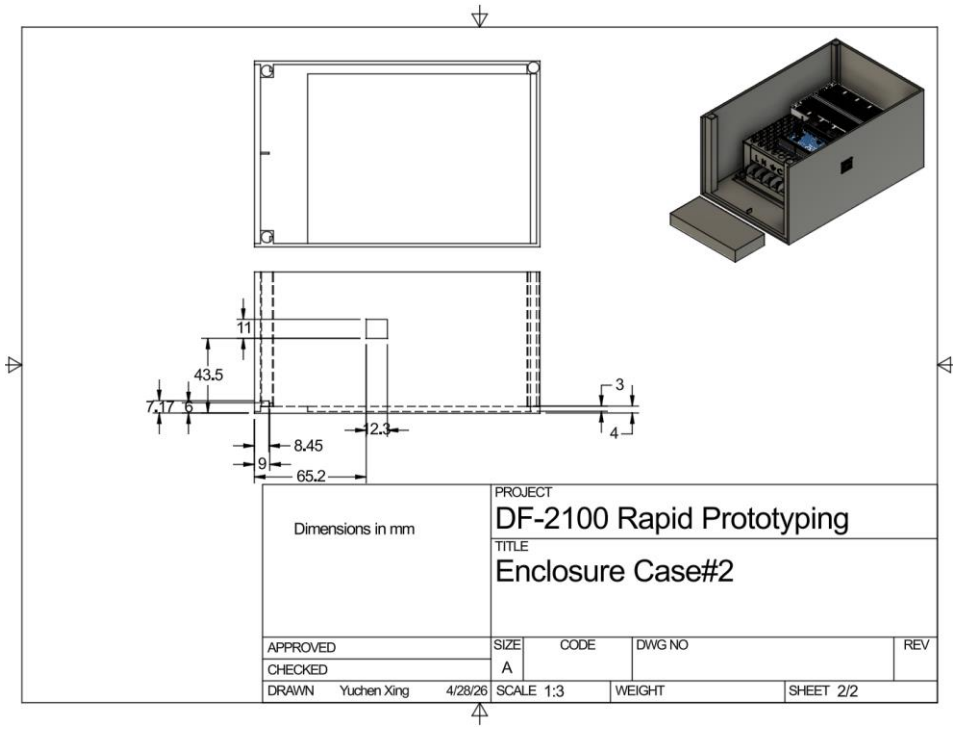

2
9
22
9
2
2
2X Ø5.8 THRU
Ø10 ↧ 5.7
18
12
20
Ø16.4 THRU
83.15
2xØ1.8 ↧6
9.55
Ø8.2 THRU
50
20
30.45
18
8
20
32
40
PROJECT
DF-2100 Rapid Prototyping
TITLE
New End-Support
APPROVED
CHECKED
DRAWN Yuchen Xing 4/28/26
SIZE
A
CODE
DWG NO
REV
SCALE 1:1
WEIGHT
SHEET 1/1


3.7
5.8
2.48
2.48
4.47
2.38
7.92
8.03
10.4
A
10
39.8
39.8
7.25
5.75
7.25
7.25
30.43
166.5
117.07
3.75
3.75
31.73
110
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
lid
APPROVED
CHECKED
DRAWN Yuchen Xing 4/28/26
SIZE
A
CODE
DWG NO
REV
SCALE 1:1
WEIGHT
SHEET 1/1


96.6
94.6
9.7
7.7
80.9
77.7
70.43
67.28
39.53
31
18
Ø12.05 THRU
14.61
16.43
21.76
30.42
51.26
69.03
76.49
83.94
89.7
104.3
4xØ3.33 THRU
Ø7.02 THRU
2xØ10.12 THRU
2xØ15.4 THRU
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
Modular Wall
APPROVED
CHECKED
DRAWN Yuchen Xing 4/29/26
SIZE A
CODE
DWG NO
REV
SCALE 1:2
WEIGHT
SHEET 1/2


9.7
7.7
3.3
11.4
13.4
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
Modular Wall
APPROVED
CHECKED
DRAWN Yuchen Xing 4/29/26
SIZE
A
CODE
DWG NO
REV
SCALE 1:1
WEIGHT
SHEET 2/2


162.75
7.25
2
104.75
108.5
101.01
106.5
54.25
1
5.75
3xØ6.5↧7.8
3.5
2
11
31
159
161
164.5
166.5
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
Enclosure Case#2
APPROVED
CHECKED
DRAWN Yuchen Xing 4/28/26
SIZE A
CODE
DWG NO
REV
SCALE 1:3
WEIGHT
SHEET 1/2


11
43.5
7.17
6
3
4
2.3
8.45
9
65.2
Dimensions in mm
PROJECT
DF-2100 Rapid Prototyping
TITLE
Enclosure Case#2
APPROVED
CHECKED
DRAWN Yuchen Xing 4/28/26
SIZE
A
CODE
DWG NO
REV
SCALE 1:3
WEIGHT
SHEET 2/2

Bibliography

1. *What is a Syringe Pump? Learn The Types, Functions, & Uses. (2018, March 29). Retrieved from Chemyx website: https://chemyx.com/resources/knowledge-base/general-syringe-pump-info/what-is-a-syringe-pump-3/*